

UART Serial Port Programming

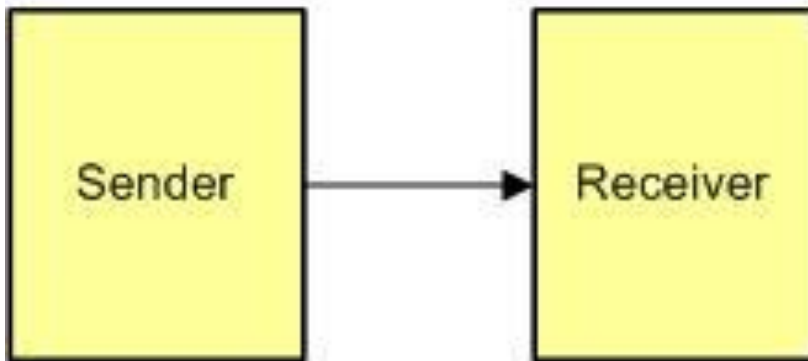
Winter 2025

Topics

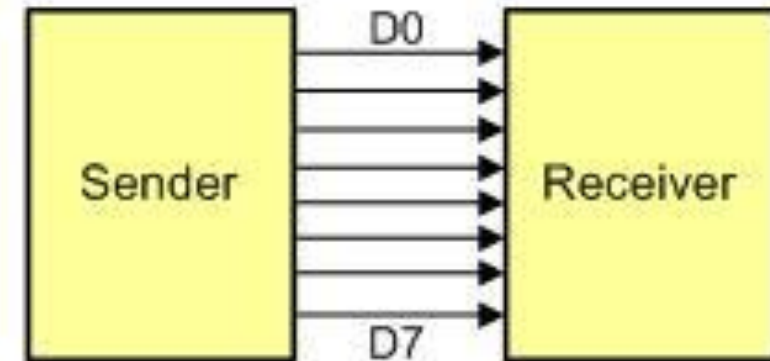
- Basics of Serial Communication
 - Half- and full-duplex transmission
 - Asynchronous serial communication and data framing
 - RS232 and other serial I/O standards
 - Data Communication Classification
- Programming UART Ports

Serial vs. Parallel Data Transfer

Serial Transfer

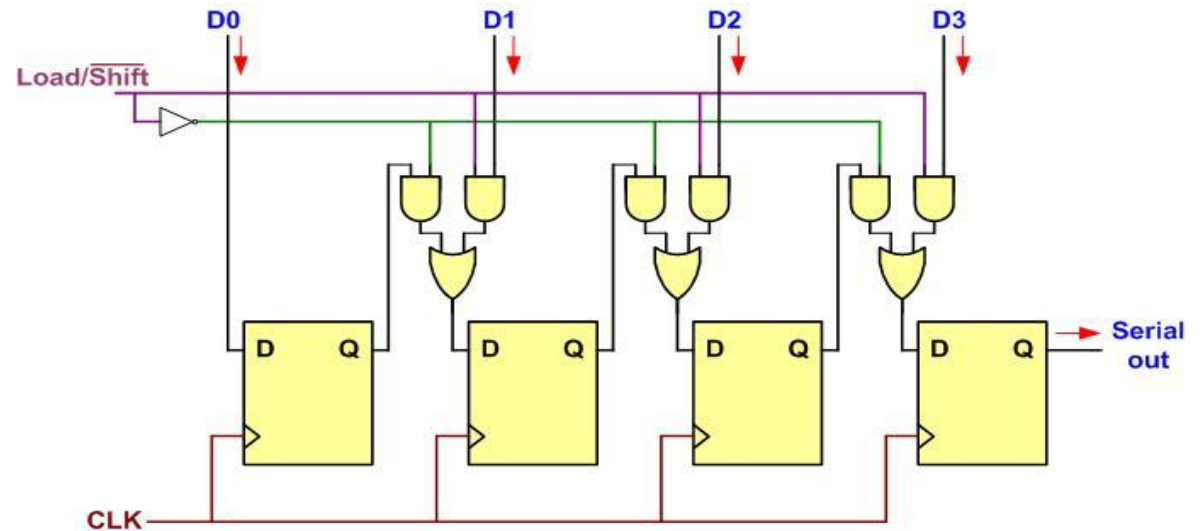


Parallel Transfer



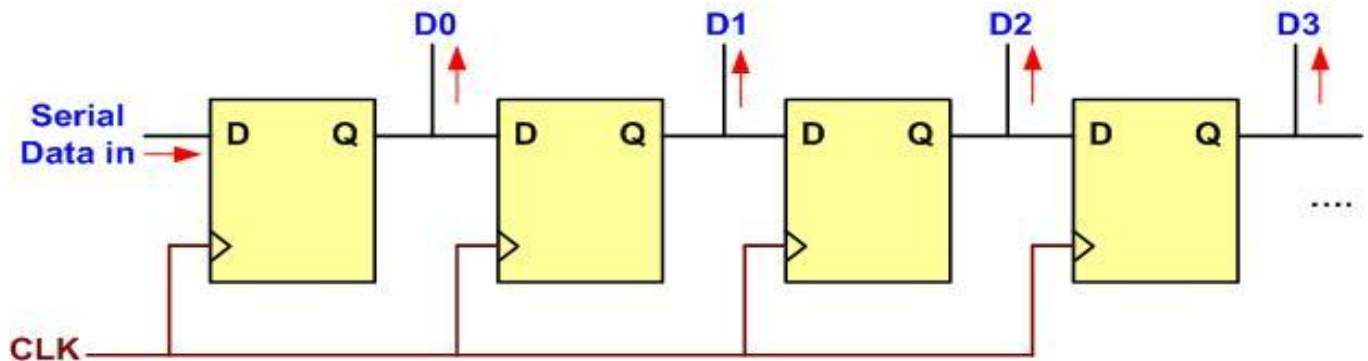
Parallel In Serial Out

For serial data communication to work, the byte of data must be grabbed from the 8-bit data bus of the microprocessor and converted to serial bits using a parallel-in-serial-out shift register on the serial-out side.

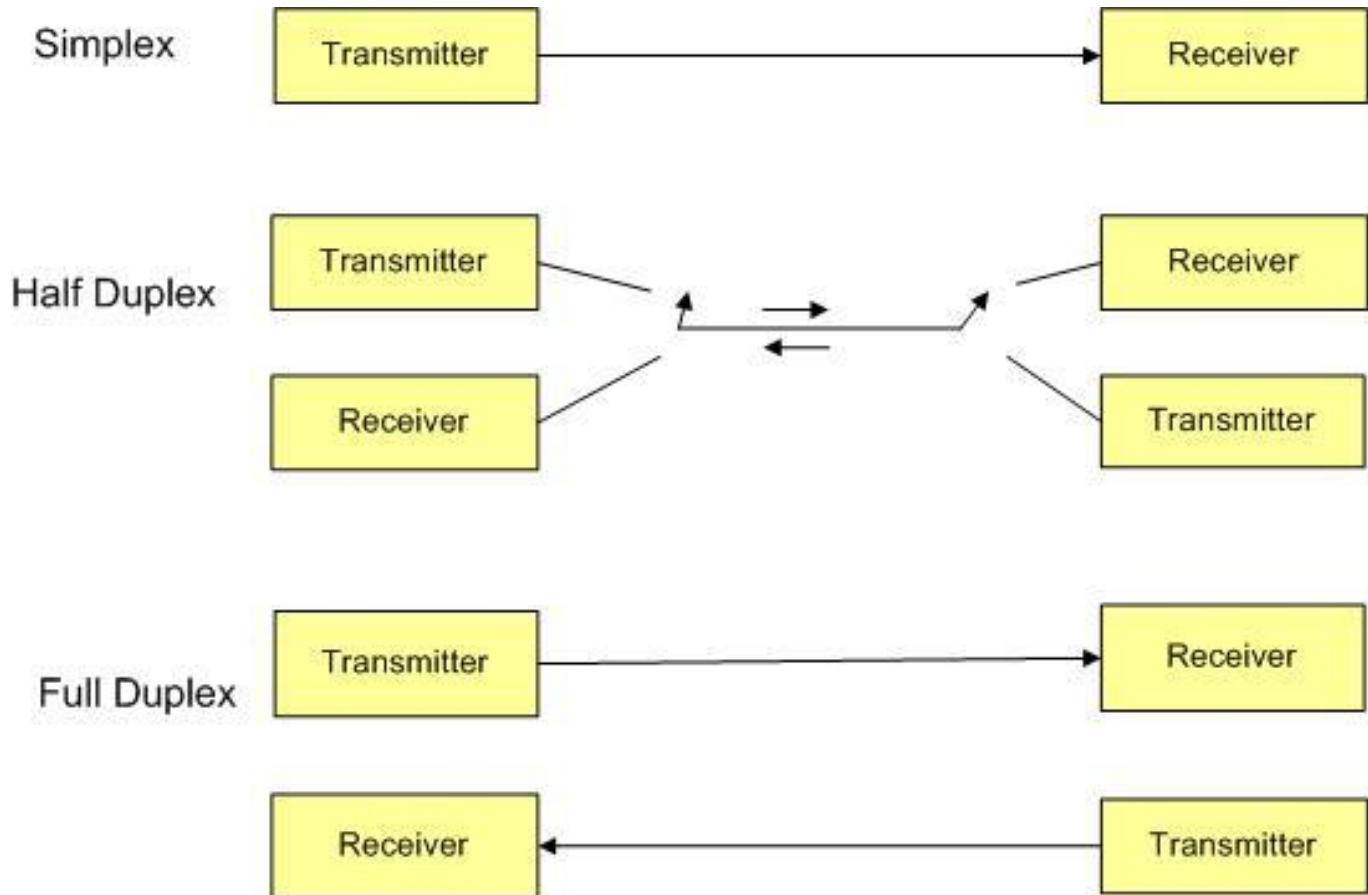


Serial In Parallel Out

At the receiving end, a serial-in-parallel-out shift register will receive the serial data, pack it into a byte, and present it to the system at the receiving end.



Simplex, Half-, and Full-Duplex Transfers



Protocol Layer

Asynchronous serial communication & data framing

- When data is transmitted serially, the receiver only sees a stream of **1s and 0s**.
- Without predefined rules, this stream is meaningless.
- To correctly interpret the data, both sender and receiver must agree on a **communication protocol**.

Communication Protocol

- The Protocol Defines:
 - How data is formatted (framed)
 - How many bits represent one character
 - When transmission starts
 - When transmission ends
 - The transmission speed (baud rate)
 - Whether error checking is used (parity)

Key Concept

- In asynchronous communication, **timing and framing replace the need for a shared clock line.**

UART Frame Format

1 start bit	5-8 data bits	0 to 1 parity bit (Optional)	1 to 2 stop bit
-------------	---------------	---------------------------------	-----------------

Component	Description
Start Bit	1 bit (LOW)
Data Bits	5–8 bits
Parity	Optional
Stop Bit	1 or 2 bits

Example: **9600 8N1**

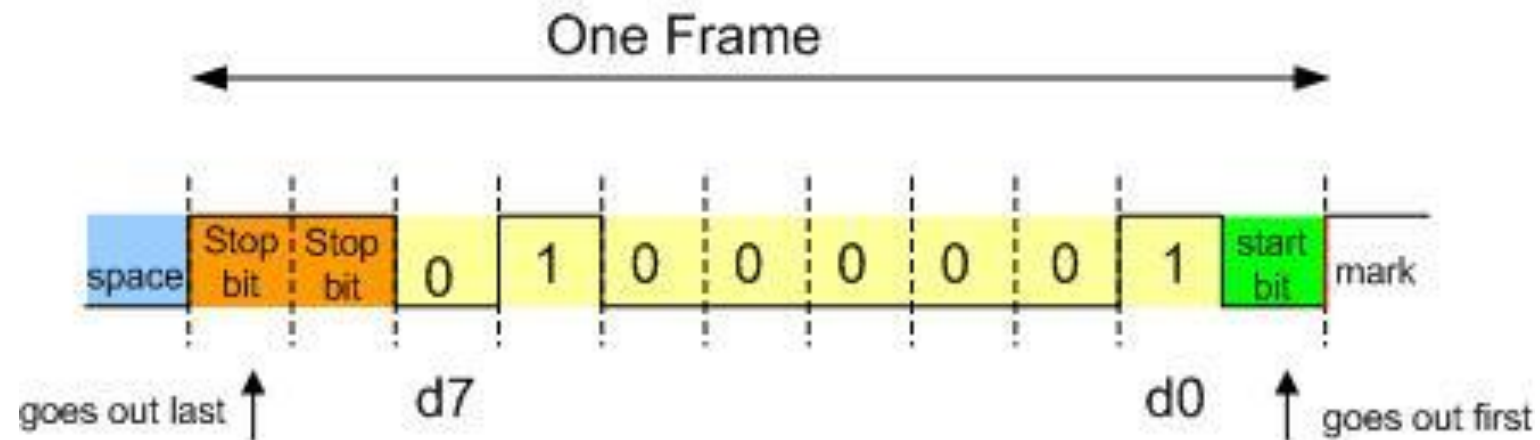
- 9600 baud
- 8 data bits
- No parity
- 1 stop bit

Start and Stop Bits

Asynchronous serial data communication is widely used for character-oriented transmissions.

In asynchronous mode, each character (the ASCII code for the character) is packed between **start and stop bits**. This is called **framing**.

Example: Framing ASCII "A" (0x41)



Parity Bit

- To improve **data integrity**, a parity bit may be added to each transmitted character.
- When parity is enabled, the UART frame contains:
 - 1 Start bit
 - 5–8 Data bits
 - **1 Parity bit (optional)**
 - 1 or 2 Stop bits
- This means each character can include **one additional bit** used for simple error detection.

How Parity Works

- The parity bit is calculated based on the number of 1s in the data bits.
- **Even Parity:** The total number of 1s (data + parity) is **even**.

Data: 10111010 → (5 ones)
Parity: 1 → Total 6 (even)

Data: 10110010 → (4 ones)
Parity: 0 → Total still 4 (even)

- **Odd Parity**

- The total number of 1s (data + parity) is **odd**.

Data: 10111010 → (5 ones)
Parity: 0 → Total still 5 (odd)

Data: 10110010 → (4 ones)
Parity: 1 → Total still 5 (odd)

Data Transfer Rate

- The data transfer rate in serial data communication is stated in **bps (bit per second)**.
- A widely used terminology for bps is **baud rate**.

Baud Rate (bps)	Typical Use
300	Legacy systems
1200	Early modems
2400	Low-speed devices
4800	Embedded systems
9600 ★	Very common default
14400	Modems
19200	Sensors, GPS
38400	Industrial devices
57600	Faster debugging
115200 ★	Very common (MCU debugging)
230400	High-speed serial
460800	Advanced systems
921600	Very high-speed UART

Electrical Layer

Everything we discussed so far defines the data format.
But how are these bits physically transmitted on a wire?

RS232 Standard

- RS232 (Recommended Standard 32) is a serial communication standard that defines how data is transmitted between devices over a **single wire**.
- It was originally developed for connecting computers to peripherals like modems and printers.

What RS-232 Defines

- RS-232 specifies:
 - Voltage levels
 - Signal timing
 - Pin configuration
 - Cable length
 - Communication direction
- It does **NOT** define the data format itself (that is handled by UART).

Where is RS232 Used Today?

- **Industrial Equipment** – PLCs, CNC machines, and robotics.
- **Embedded Systems** – Microcontrollers, IoT, and debugging.
- **Point-of-Sale Systems** – Barcode scanners and receipt printers.
- **Old Modems and Telecommunication Equipment.**

Electrical Characteristics

- Unlike TTL UART (0V–3.3V or 0V–5V), RS-232 uses higher and inverted voltage levels:

Logic Level	RS-232 Voltage	UART in Microcontroller
Logic 1 (Mark)	–3V to –15V	5V or 3.3V
Logic 0 (Space)	+3V to +15V	0

Note:

- RS-232 logic is **inverted**
- It uses **negative voltage for logic 1**



Microcontrollers like TM4C123 uses logic level (0-3.3V).

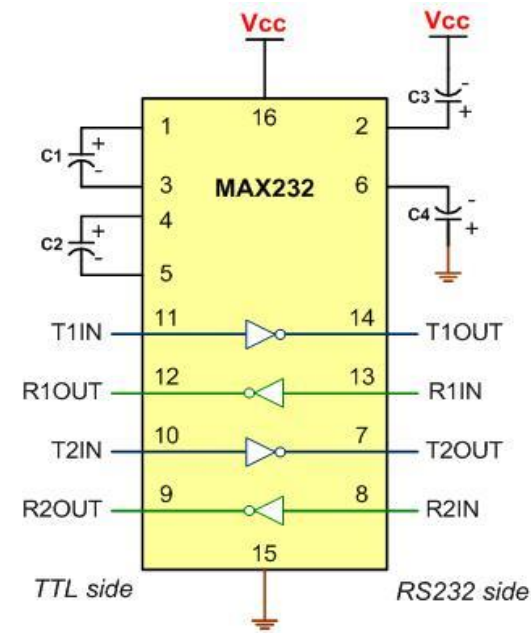
We need a **Level Shifter**.

MAX232– RS-232 to TTL Level Converter

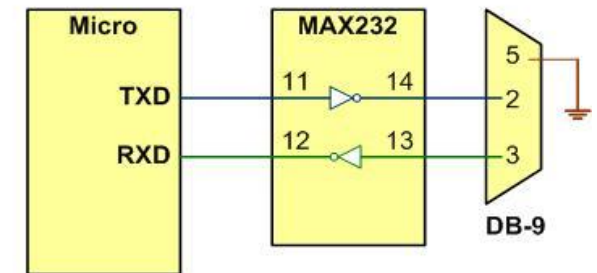
MAX232 is an integrated circuit (IC) that converts:

RS-232 voltage levels ($\pm 12\text{V}$) $\leftrightarrow$ TTL logic levels (0V–3.3V)

It performs **both voltage level shifting and signal inversion**.

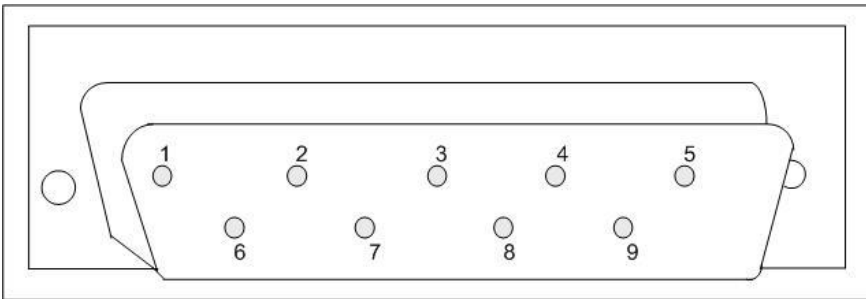


(a) Inside MAX232



(b) MAX232 Connection to the Microcontroller / Microprocessor

RS232 Pins and 9-Pin Male Connector



Pin	Description
1	Data carrier detect (DCD)
2	Received data (RxD)
3	Transmitted data (TxD)
4	Data terminal ready (DTR)
5	Signal ground (GND)
6	Data set ready (DSR)
7	Request to send (RTS)
8	Clear to send (CTS)
9	Ring indicator (RI)

DTE and DCE

- In RS-232 communication, devices are categorized as:
 - **DTE (Data Terminal Equipment)**
 - **DCE (Data Circuit-terminating Equipment)**
- This classification determines **signal direction** and **cable wiring**.

DTE (Data Terminal Equipment)

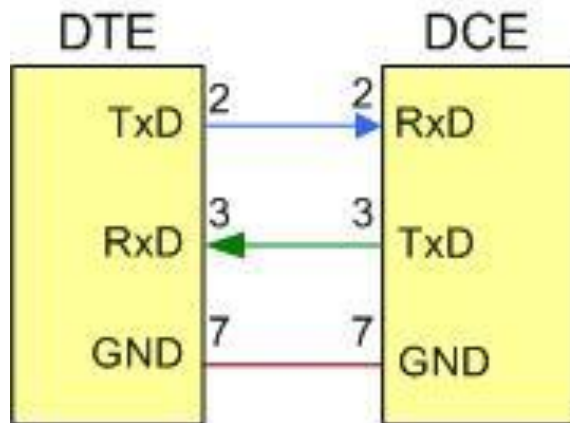
- These are **end-user devices** that:
 - Generate data
 - Consume data
- **Examples:**
 - Computer (PC)
 - Microcontroller board (e.g., TM4C123)
 - Printer
 - Terminal
- DTE devices typically transmit data on **TX pin** and receive on **RX pin**.

DCE (Data Circuit-terminating Equipment)

- These devices:
 - Provide communication interface to the transmission medium
 - Connect DTE devices to communication lines
- **Examples:**
 - Modem
 - Some network interface devices
 - Industrial communication equipment
- DCE devices have opposite pin directions compared to DTE.

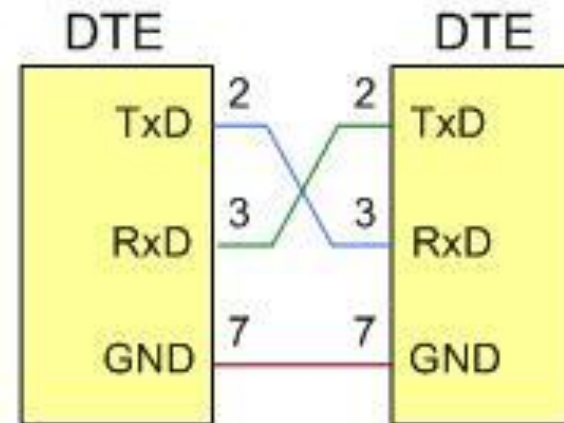
DTE-DCE and DTE-DTE Connections

DTE- DCE Connection



Can connect directly (**straight cable**)

DTE- DTE Connection



Needs a **null modem cable** (TX/RX crossed)

TM4C123 ↔ PC (real RS-232)

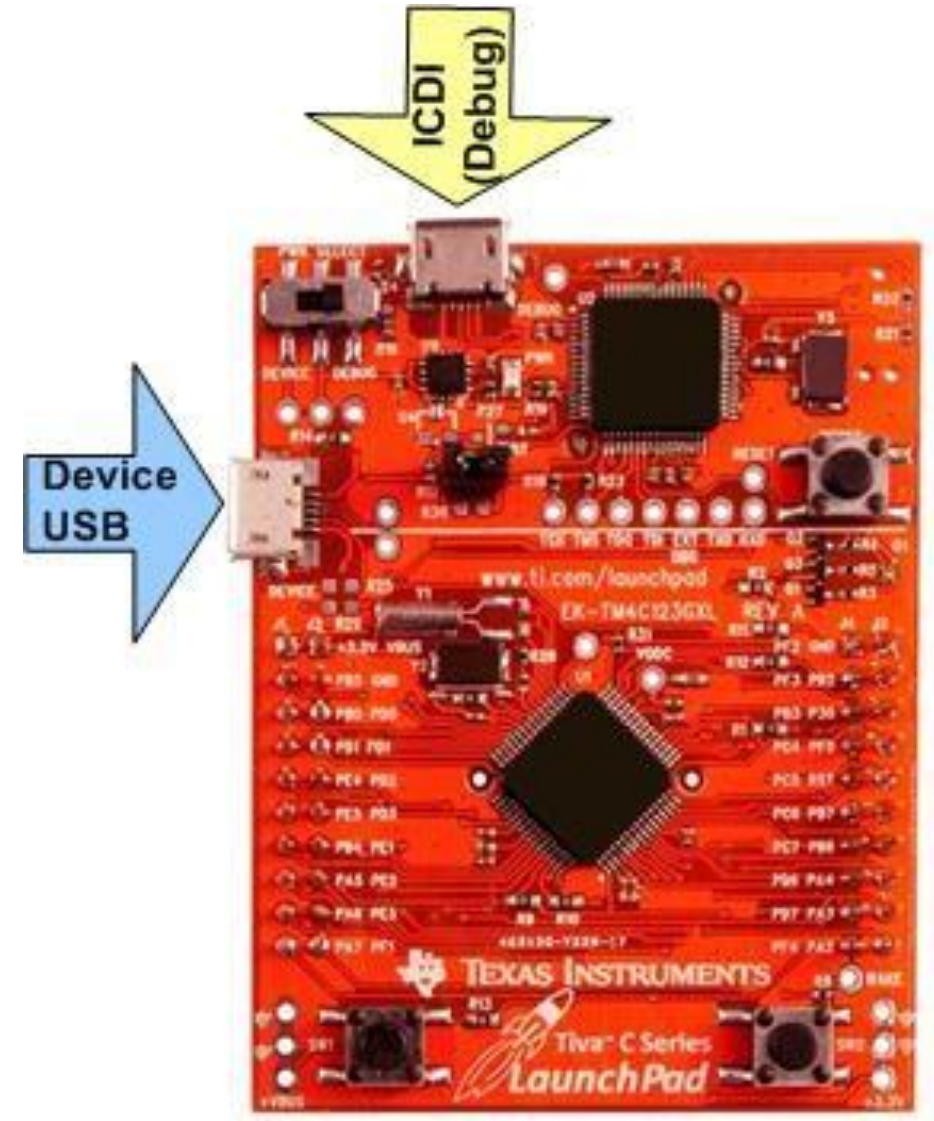
UART to USB

- **UART-to-USB** is a method of connecting a microcontroller's UART interface to a computer's USB port using a converter chip.
- UART and USB are completely different protocols.They are not directly compatible.
- A **USB-to-UART bridge chip** performs protocol conversion.

Tiva LaunchPad

Many of the TI ARM chips come with up to **eight** on-chip UART ports: **UART0 –UART7**.

In TI LaunchPad, the UART0 port of TM4C123GH6PM is connected to the ICDI (In-Circuit Debug Interface), which is connected to a USB connector.



ICDI (In-Circuit Debug Interface)

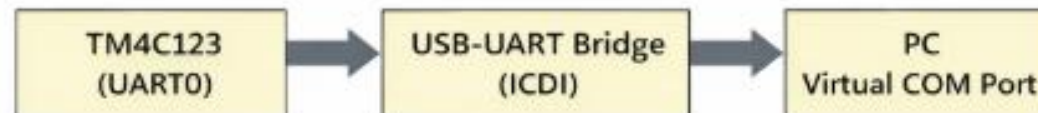
- The **ICDI** is a built-in **debugging and programming interface** on the TI LaunchPad.
- It allows your PC to:
 - Program (flash) the microcontroller
 - Debug code in real time
 - Communicate through USB

Connection Between UART0 and ICDI

What Happens on the TIVA LaunchPad?

- On the **TM4C123 LaunchPad**:
 - **UART0 (PA0/PA1)** is internally connected to the **ICDI**
 - The ICDI is connected to the **USB port**
 - This allows communication between the microcontroller and the PC

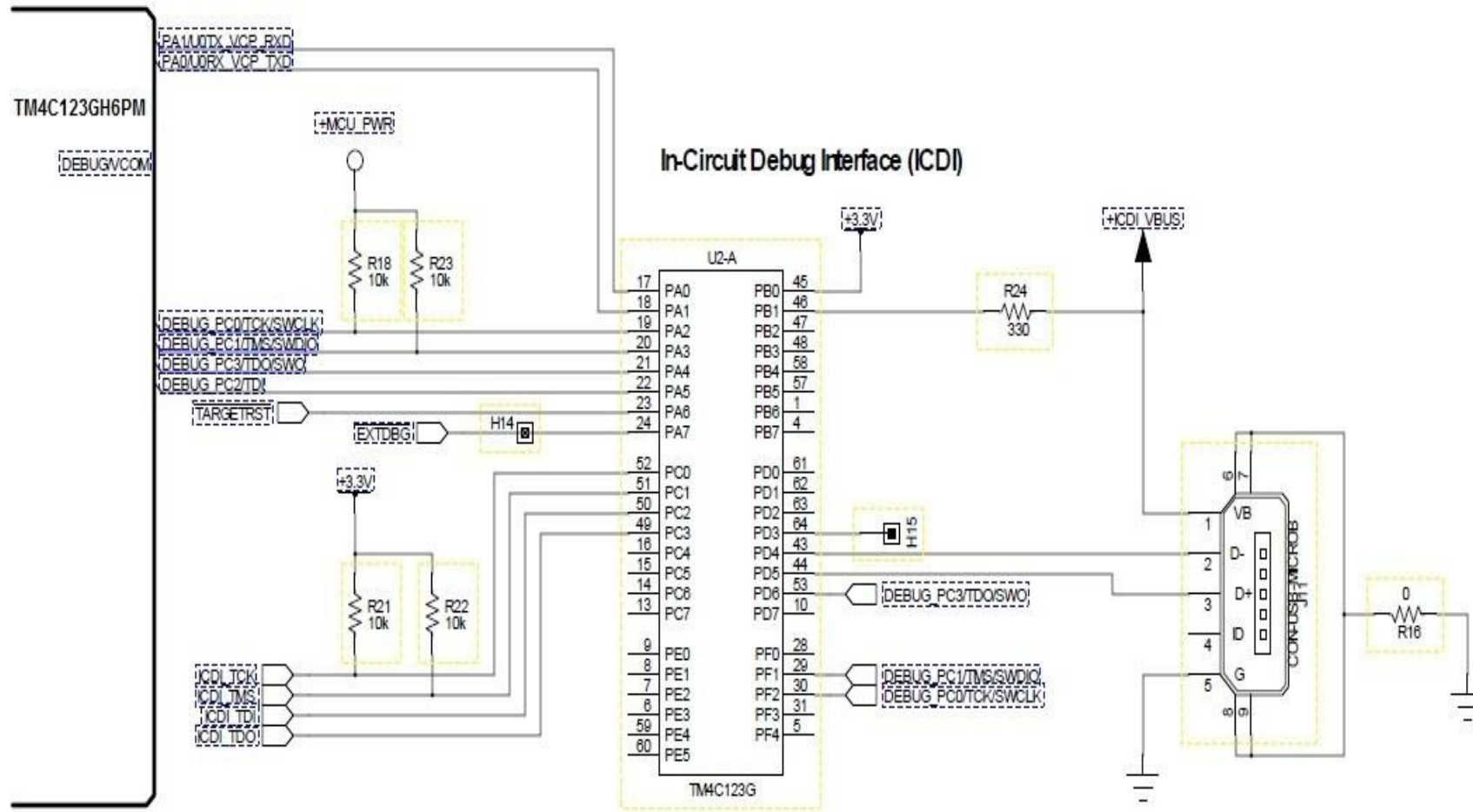
Connection Between UART0 and ICDI on TI TIVA
(TM4C123) LaunchPad



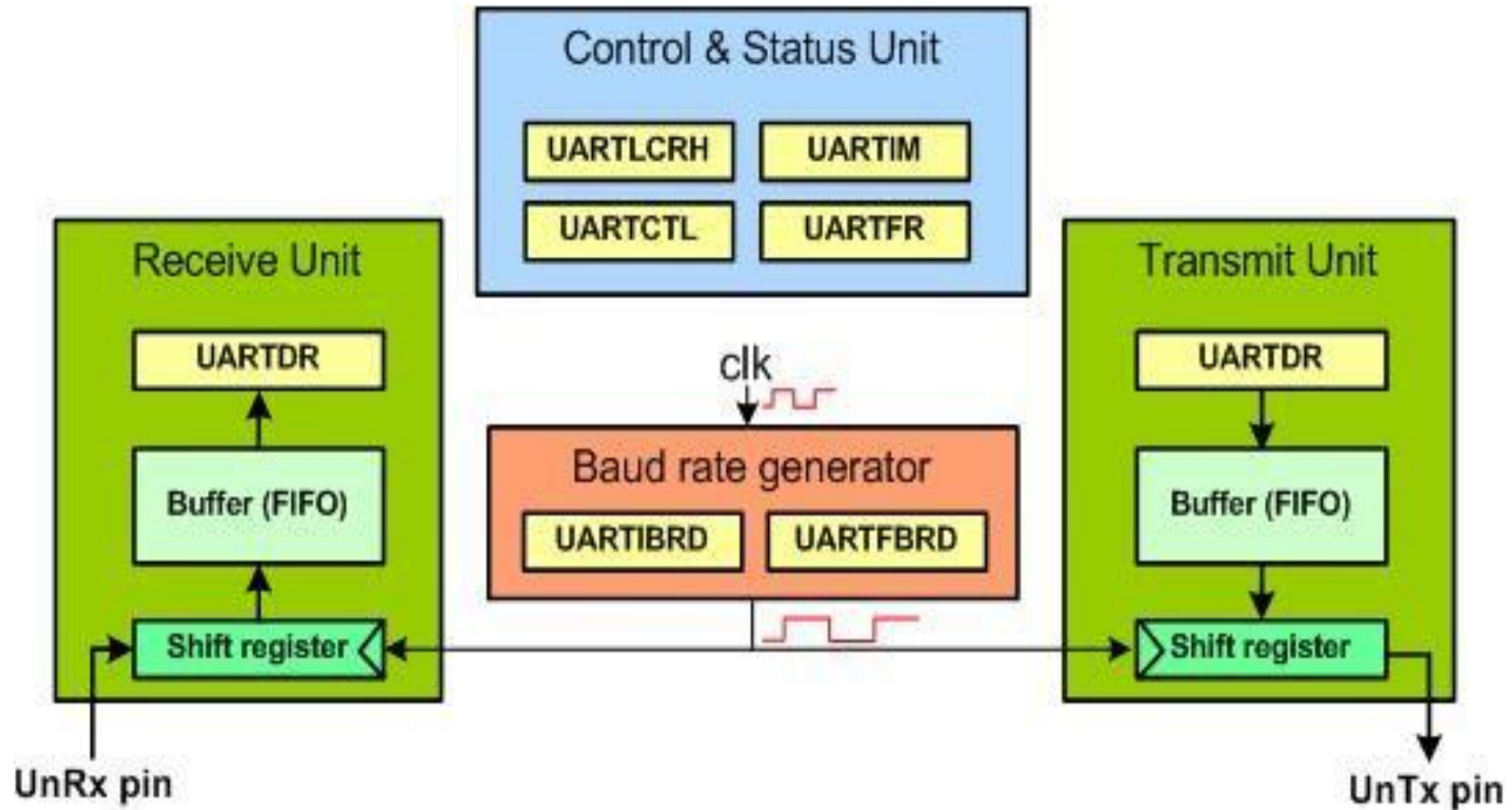
What the ICDI Actually Does

- The ICDI:
 - Receives UART0 data from the microcontroller
 - Converts UART protocol to USB protocol
 - Sends data to the PC
 - Converts incoming USB data to UART
 - Sends it back to the microcontroller
- So, it acts as a **protocol translator and bridge**.

Connection Between UART0 and ICD1



A Simplified Block Diagram of UARTn



List of Registers

- Data and Status Registers
- Baud Rate Registers
- Line Control Register
- Control Register
- Interrupt Registers
- FIFO Level Select Register
- DMA Control Register

Data & Status Registers

Register	Full Name	Purpose
UARTDR	Data Register	Read received data / Write transmit data
UARTSR / UARTECR	Receive Status / Error Clear	Framing, parity, overrun error flags
UARTFR	Flag Register	TX/RX FIFO status flags

Baud Rate Register

Register	Purpose
UARTIBRD	Integer Baud Rate Divisor
UARTFBRD	Fractional Baud Rate Divisor

Line Control Register

Register	Purpose
UARTLCRH	Line Control (Word length, Parity, Stop bits, FIFO enable)

Core Components of UARTn

TX Line (Transmitter)

- The **UnTX** line carries serial data from **the microcontroller to an external device**.
- Data written to the UART is first loaded into a **Parallel-to-Serial Shift Register**.
- The shift register transmits data **one bit at a time** onto the TX line.
- The transmission speed is controlled by the **Baud Rate Generator**.

RX Line (Receiver)

- The **UnRX** line receives serial data **from an external device**.
- Incoming data arrives **bit by bit** at the baud rate defined by the **Baud Rate Generator**.
- The data is assembled inside a **Serial-to-Parallel Shift Register**.
- Once a full character is received, it is transferred to the **Receive Buffer (FIFO)**.

Core Components of UARTn (*cntd.*)

Data Register (UARTDR) and FIFO

- The **UART Data Register (UARTDR)** is the interface between the CPU and the UART hardware.
- The UART module includes:
 - **Transmit FIFO (TX FIFO)**
 - **Receive FIFO (RX FIFO)**
- These FIFOs buffer data to improve efficiency and reduce CPU load.

Transmission Direction (TX Path)

- When transmitting data:
 - The processor writes a byte to **UARTDR**.
 - The data is placed into the **TX FIFO**.
 - When the transmitter is ready, data moves from the FIFO to the **Transmit Shift Register**.
 - The shift register sends the data **bit-by-bit** out through the **TX pin**.

Data Flow:

CPU → UARTDR → TX FIFO → Shift Register → TX Pin

Reception Direction (RX Path)

- When receiving data:
 - Serial data arrives on the **RX pin**.
 - Bits are assembled in the **Receive Shift Register**.
 - A complete character is moved into the **RX FIFO**.
 - The processor reads the data from **UARTDR** (which accesses the RX FIFO).

Data Flow:

- RX Pin → Shift Register → RX FIFO → UARTDR → CPU

Core Components of UARTn (*cntd.*)

Control and Status Unit:

- The **Control Logic** manages the overall operation of the UART, including:
 - **Start/Stop bits:** Handling the start and stop bits that frame each byte of data in asynchronous communication.
 - **Parity Bit:** If parity is enabled, the control logic generates or checks the parity bit.
 - **Interrupts:** It manages interrupts, such as when the TX FIFO is empty or the RX FIFO is full, allowing the system to know when to start or stop communication.

Core Components of UARTn (*cntd.*)

Baud Rate Generator (*Registers: UARTIBRD, UARTFBRD*)

- The **Baud Rate Generator** produces the internal clock used to:
 - Transmit bits on the TX line
 - Sample bits on the RX line
- It determines the **baud rate** (bits per second).

Baud Rate Calculation

$$\text{Desired Baud Rate} = \frac{\text{SysClk}}{16 \times \text{ClkDiv}}$$

SysClk: working system clock connected to the UART

ClkDiv: the value we load into divisor register

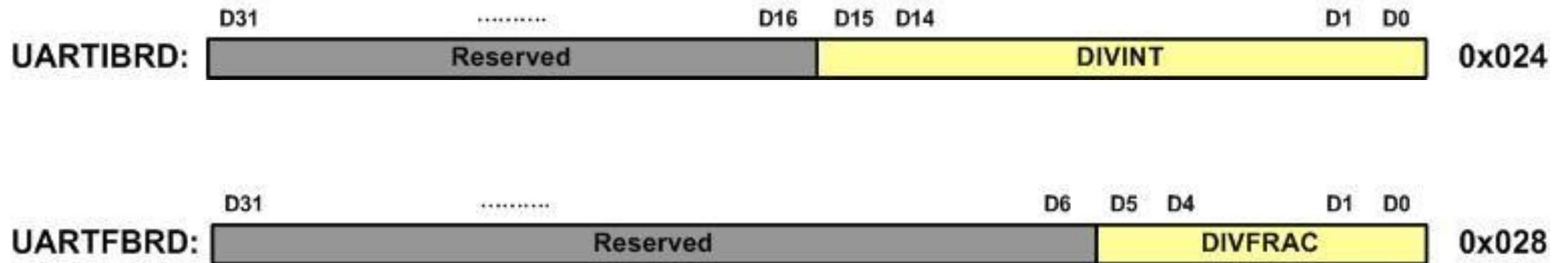
The divisor is split into:

- **UARTIBRD** → Integer part
- **UARTFBRD** → Fractional part

Baud Rate Generator (UARTIBRD, UARTFBRD)

Two registers are used to set the baud rate:

- UARTIBRD: The integer part of divisor
- UARTFBRD: The fractional part of divisor



Example (1/5)

Baud Rate Calculation for 9600 Baud with 16 MHz System Clock on Tiva C (TM4C123GH6PM)

Example (2/5)

Baud Rate Calculation for 9600 Baud with 16 MHz System Clock on Tiva C (TM4C123GH6PM)

Step 1: Compute Clock Divisor

$$Baud\ Rate = \frac{SysClk}{16 \times ClkDiv}$$

$$ClkDiv = \frac{SysClk}{16 \times Baud\ Rate}$$

$$ClkDiv = \frac{16MHz}{16 \times 9600} = \frac{16,000,000}{16 \times 9600} = 104.1667$$

Example (3/5)

Baud Rate Calculation for 9600 Baud with 16 MHz System Clock on Tiva C (TM4C123GH6PM)

Step 2: Split into Integer and Fractional Parts

Integer part=104

Fractional Part = 0.1667

Example (4/5)

Baud Rate Calculation for 9600 Baud with 16 MHz System Clock on Tiva C (TM4C123GH6PM)

Step 3: Compute Fractional Baud Rate Divisor (FBRD)

Fractional Part = 0.1667

FBRD = Fractional Part x 64+0.5

= 0.1667 x 64 + 0.5 = 11 (rounded to nearest integer)

Example (5/5)

Baud Rate Calculation for 9600 Baud with 16 MHz System Clock on Tiva C (TM4C123GH6PM)

UARTIBRD = 104

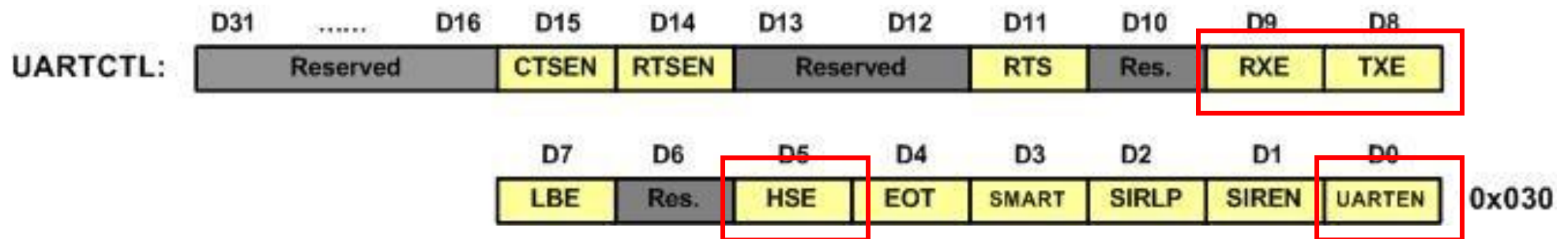
UARTFBRD = 11

Important Notes

- Both transmitter and receiver must use **the same baud rate**
- Small mismatches cause:
 - Framing errors
 - Garbage characters
- Baud generator also controls RX sampling timing

UART Control (UARTCTL)

UARTCTL is a 32-bit register and many of the bits are unused. For us, the most important bits are RXE, TXE, HSE, and UARTEN.



UART Control (UARTCTL)

UARTEN (D0) UART Enable:

- This allows one to enable/disable the UART.

HSE (D5) High Speed Enable:

- For the baud rate divisor, by default the system clock is divided by 16 before it is fed to the UART. We can change it to divide-by-8 by setting HSE = 1 to run higher baud rate with a low system clock frequency.

RXE (D8) Receive Enable:

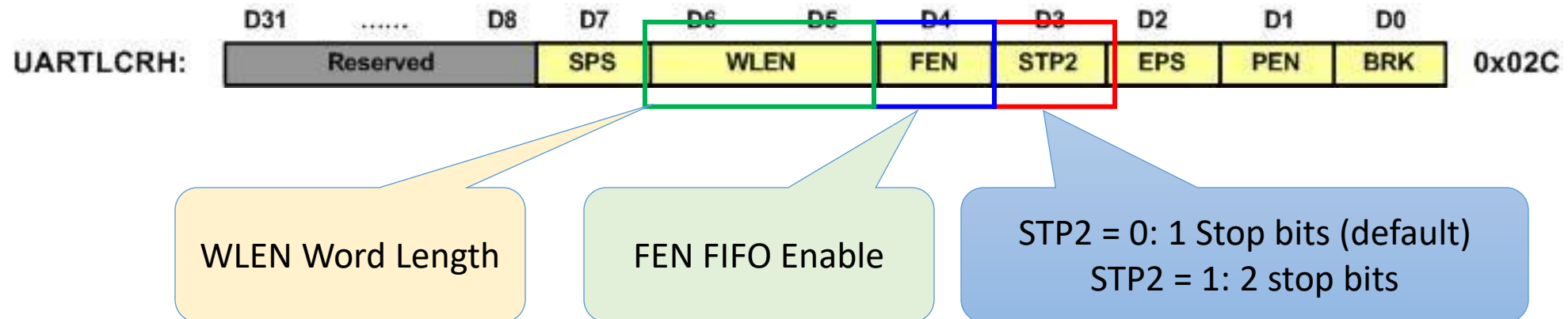
- We must enable this bit to receive data.

TXE (D9) Transmit Enable:

- We must enable this bit to sent data.

UART Line Control (UARTLCRH)

This register is used to set the number of bits per character (data length) in a frame and number of stop bits among other things.



Review: UART Frame Format

1 start bit	5-8 data bits	0 to 1 parity bit (Optional)	1 to 2 stop bit
-------------	---------------	---------------------------------	-----------------

Component	Description
Start Bit	1 bit (LOW)
Data Bits	5–8 bits
Parity	Optional
Stop Bit	1 or 2 bits

Example: **9600 8N1**

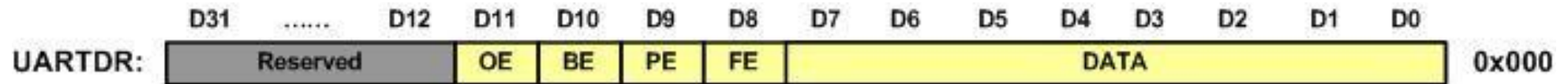
- 9600 baud
- 8 data bits
- No parity
- 1 stop bit

WLEN Word Length

D6	D5	
0	0	5 bits
0	1	6 bits
1	0	7 bits
1	1	8 bits

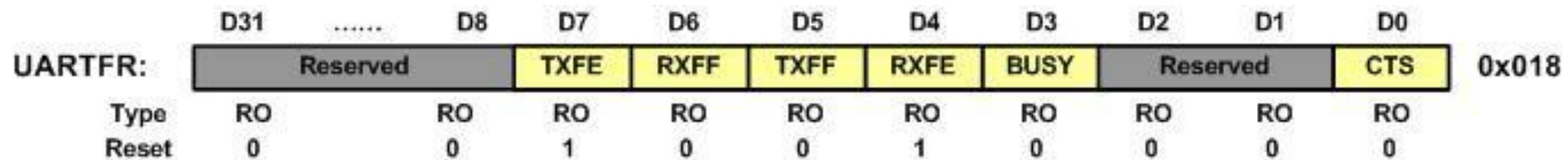
UART Data (UARTDR)

We must place data in UART Data Register (UARTDR) to transmit a byte of data.



UART Flag Register (UARTFR)

The **UART Flag Register (UARTFR)** provides status flags for the UART module. It indicates whether the UART is **busy**, **ready to send/receive data**, or has encountered an **error**.



UART Flag Register (UARTFR)

Bit	Name	Meaning
0	CTS	Clear to Send (1 = not ready, 0 = ready)
1	DSR	Data Set Ready (for modem control)
2	DCD	Data Carrier Detect (for modem control)
3	BUSY	UART Busy (1 = TX/RX ongoing, 0 = idle)
4	RXFE	Receive FIFO Empty (1 = no data, 0 = data available)
5	TXFF	Transmit FIFO Full (1 = full, 0 = can send)
6	RXFF	Receive FIFO Full (1 = full, 0 = can receive)
7	TXFE	Transmit FIFO Empty (1 = empty, 0 = data in FIFO)
8-31	-	Reserved

RCGCUART

The **RCGCUART (Run Mode Clock Gating Control UART) Register** is used to enable or disable the clock for the **UART modules** in the Tiva C microcontroller.



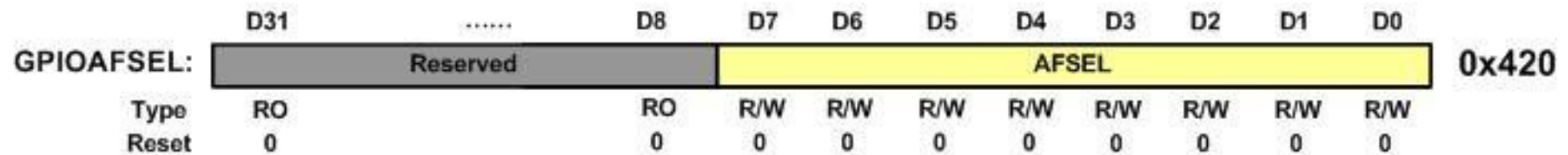
Each bit in this register corresponds to one of the **eight** UART modules available in Tiva C:

Bit	UART Module	Description
0	UART0	Enables clock for UART0
1	UART1	Enables clock for UART1
2	UART2	Enables clock for UART2
3	UART3	Enables clock for UART3
4	UART4	Enables clock for UART4
5	UART5	Enables clock for UART5
6	UART6	Enables clock for UART6
7	UART7	Enables clock for UART7

GPIO Alternate Function Select (GPIOAFSEL)

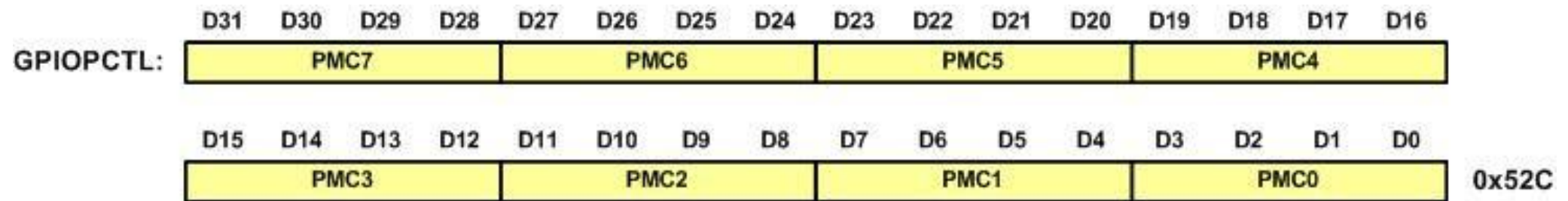
The GPIOAFSEL (GPIO Alternate Function Select) Register enables or disables the alternate function mode for GPIO pins.

- **Setting a bit to 1** → The pin is used for an alternate function (UART, SPI, I2C, PWM, etc.).
- **Setting a bit to 0** → The pin works as a regular GPIO (input or output).



GPIOPCTL Register

The **GPIOPCTL (GPIO Port Control) Register** is used to select the **peripheral function** for GPIO pins in **Tiva C**. It configures **which alternate function** (like UART, SPI, I2C, etc.) a GPIO pin will use.



Each GPIO pin is controlled by **4 bits** in GPIOPCTL.

62

Table 10-11. GPIO Pins With Special Considerations

GPIO Pins	Default Reset State	GPIOAFSEL	GPIODEN	GPIOPDR	GPIOPUR	GPIOCTL	GPIOCR
PA[1:0]	UART0	0	0	0	0	0x1	1
PA[5:2]	SSI0	0	0	0	0	0x2	1
PB[3:2]	I ² C0	0	0	0	0	0x3	1
PC[3:0]	JTAG/SWD	1	1	0	1	0x1	0
PD[7]	GPIO ^a	0	0	0	0	0x0	0
PF[0]	GPIO ^a	0	0	0	0	0x0	0

a. This pin is configured as a GPIO by default but is locked and can only be reprogrammed by unlocking the pin in the **GPIOLOCK** register and uncommitting it by setting the **GPIOCR** register.

How to Use GPIOPCTL?

- **1. Find the Function Code for a Pin**
- Each **alternate function** has a unique **function code** (Table in TM4C123 datasheet).

Function	Code
GPIO (Default)	0x0
UART	0x1
SSI (SPI)	0x2
I2C	0x3
PWM	0x4
CAN	0x8
USB	0xC

Steps to Configure UART in Tiva C

1. Enable the UART and GPIO Clock
2. Configure GPIO Pins for UART (Alternate Function)
3. Disable UART Before Configuration
4. Set the Baud Rate (IBRD and FBRD Calculation)
5. Set Data Format (Word Length, Stop Bits, Parity)
6. Enable UART and Enable TX & RX
7. Transmit and Receive Data

Step1: Enable the UART and GPIO Clock

Example: Enable the UART0 and GPIOA Clock

- `SYSCTL->RCGCUART |= 1; /* provide clock to UART0 */`
- `SYSCTL->RCGCGPIO |= 1; /* enable clock to PORTA */`

Step 2: Configure GPIO Pins for UART (Alternate Function)

Example: Configure PA0 & PA1 as UART0 Pins

- Set Alternate Function (AFSEL)
- Enable Digital Function (DEN)
- Configure Control (PCTL) for UART Mode

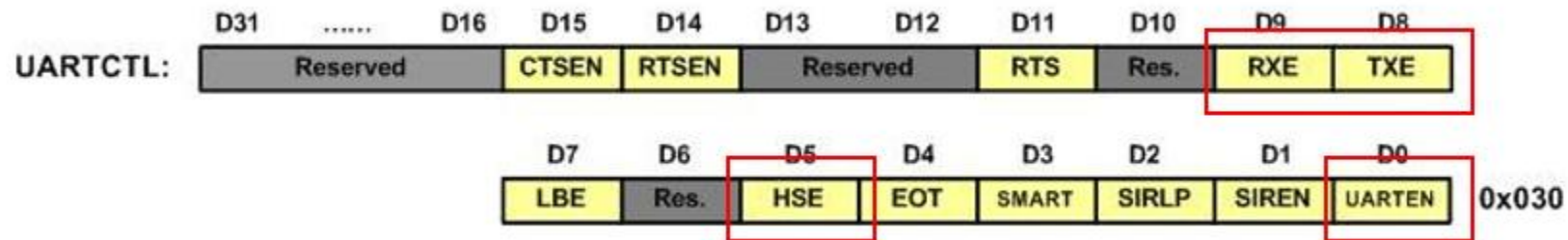
```
GPIOA->DEN = 0x03;          /* Make PA0 and PA1 as digital */
```

```
GPIOA->AFSEL = 0x03;        /* Use PA0,PA1 alternate function */
```

```
GPIOA->PCTL = 0x11;         /* configure PA0 and PA1 for UART */
```

Step 3- Disable UART Before Configuration

```
UART0->CTL = 0;          /* disable UART0 */
```



Step 4- Set Baud Rate (9600 Baud, 16MHz Clock)

- **Calculate IBRD & FBRD for 9600 baud**
 - **System Clock = 16 MHz**
 - **IBRD = 104**
 - **FBRD = 11**

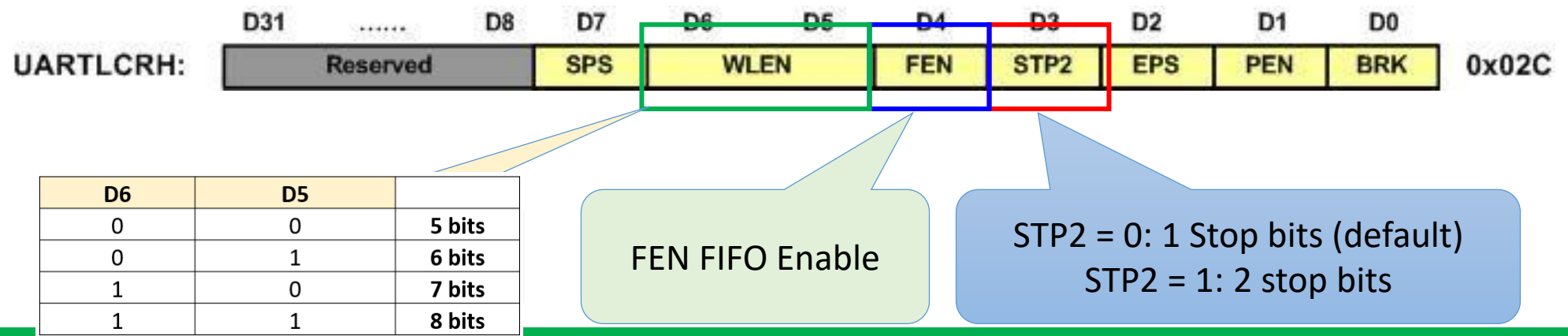
```
UART0->IBRD = 104;      /* 16MHz/16=1MHz, 1MHz/104=9600 baud rate */
```

```
UART0->FBRD = 11;      /* fraction part*/
```

Step 5- Configure UART Line Control

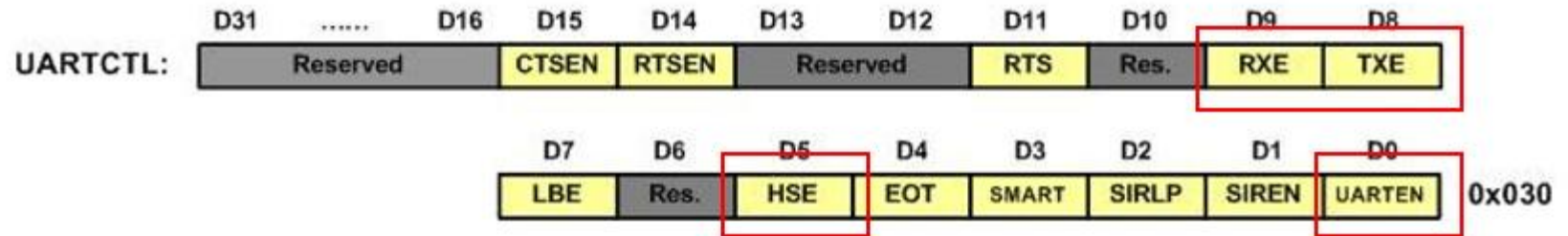
Example: (8-bit, No Parity, 1 Stop Bit, No FIFO)

```
/* 8-bit, no parity, 1-stop bit, no FIFO */  
UART0->LCRH = 0x60;
```



Step 6- Enable UART, TX & RX

```
UART0->CTL = 0x301;    /* enable UART0, TXE, RXE */
```



Step 7: Sending Data

```
/* UART0 Transmit */
/* This function waits until the transmit buffer is available then */
/* writes the character in the transmit buffer. It does not wait */
/* for transmission to complete. */

void UART0Tx(char c)
{
    while((UART0->FR & 0x20) != 0); /* wait until Tx buffer not full */
    UART0->DR = c;                  /* before giving it another byte */
}
```

Checking UARTFR's TX Buffer

Bit	Name	Meaning
0	CTS	Clear to Send (1 = not ready, 0 = ready)
1	DSR	Data Set Ready (for modem control)
2	DCD	Data Carrier Detect (for modem control)
3	BUSY	UART Busy (1 = TX/RX ongoing, 0 = idle)
4	RXFE	Receive FIFO Empty (1 = no data, 0 = data available)
5	TXFF	Transmit FIFO Full (1 = full, 0 = can send)
6	RXFF	Receive FIFO Full (1 = full, 0 = can receive)
7	TXFE	Transmit FIFO Empty (1 = empty, 0 = data in FIFO)
8-31	-	Reserved

Step 7- Receiving Data

```
/* UART0 Receive */
/* This function waits until a character is received then returns it. */

char UART0Rx(void)
{
    char c;
    while((UART0->FR & 0x10) != 0); /* wait until the RX buffer is not empty */
    c = UART0->DR; /* read the received data */
    return c; /* and return it */
}
```

Checking UARTFR's RX Buffer

Bit	Name	Meaning
0	CTS	Clear to Send (1 = not ready, 0 = ready)
1	DSR	Data Set Ready (for modem control)
2	DCD	Data Carrier Detect (for modem control)
3	BUSY	UART Busy (1 = TX/RX ongoing, 0 = idle)
4	RXFE	Receive FIFO Empty (1 = no data, 0 = data available)
5	TXFF	Transmit FIFO Full (1 = full, 0 = can send)
6	RXFF	Receive FIFO Full (1 = full, 0 = can receive)
7	TXFE	Transmit FIFO Empty (1 = empty, 0 = data in FIFO)
8-31	-	Reserved

Reference

1. Mazidi, M., Chen, S., Naimi, S., Naimi, S., Chen, S. (2016). TI TIVA ARM Programming For Embedded Systems, 1st Edition. MicroDigitalEd. ISBN-13: 978-0997925920
2. Valvano, J., (2019). Embedded Systems: Introduction to ARM Cortex-M Microcontrollers, 5th edition, 6th printing